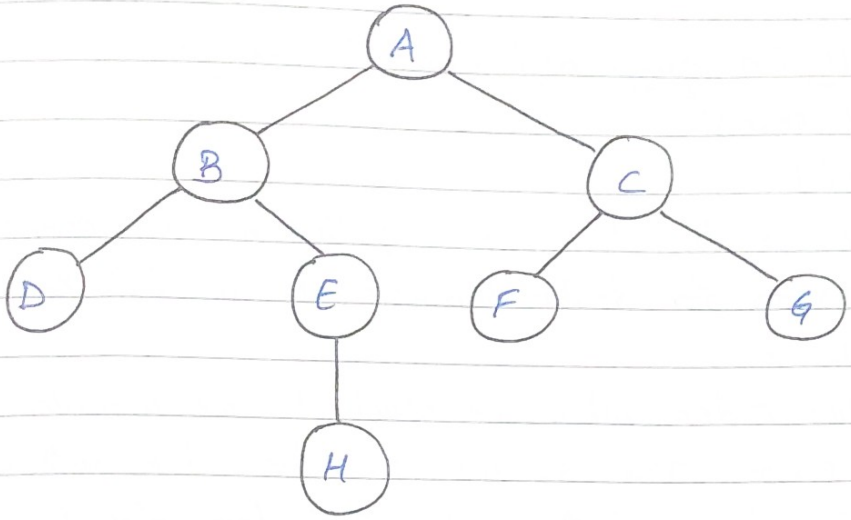
# Lab-2a Breadth First Search (BFS)

To implement the BFS algorithm to find the shortest path from the start node to a goal node in a graph.




we need to find a path from A to H using the BFS algorithm.

Step 1: Declare the graph as a dictionary

```
graph = {
        'A' : ['B', 'C'],
        'B' : ['D', 'E'],
        'C' : ['F', 'G'],
        'D' : [] , 'E' : ['H'],
        'F' : [], 'G' : [], 'H' : []}
```

This means
A is connected to B and C
B is connected to D and E
C is connected to F and G
E is connected to H

Nodes D, F, G, H are leaf nodes, they don't have any children. That is why they get an empty [] list.

Step 2: Declare the BFS function

def bfs(start, goal):

creates a function that searches from the start node to the goal node. We can reuse the function for any query since the parameters are not hard coded.

Step 3: Declare queue and visited variables

queue = [[start]]

visited = []

The queue variable is holding paths not the nodes, that is the reason for double brackets, its a list containing another list.

If it holds nodes, when we reach H we wouldnt know how we reached there so we store the whole paths.

visited=[] stores all the nodes that have already been explored.

Step 4: Open the loop and begin from front

```
while queue:
    path = queue.pop(0)
    node = path[-1]
```

Continue searching until the queue becomes empty.

queue.pop(0): removes the first item i.e item at index 0, the front.

This line ~~makes~~ is what makes it BFS.

path[-1] : gives the last element of the list

Ex: path = ['A', 'B', 'E']

node = E

Step 5: Goal test

```
if node == goal:
    return path, visited
```

This will give the path and visited nodes when we reach the goal.

Step 6: Avoid revisiting

```
if node not in visited:
    visited.append(node)
```

This stops a node from being expanded twice when two paths which are different reach the same node.
Append the node if it is not yet visited.

Step 7: Explore neighbours

```
for n in graph[node]:
    queue.append(path + [n])
```

search for every neighbour of the node.
path + [n] creates a new list. It does not modify path.

Step 8: Return failure if queue empties

```
return None, visited
```

If there doesnt exist any path return none and also the visited nodes.

Step 9: Call and Print

```
p, v = bfs('A', 'H')
print('BFS path :", p)
print(" BFS visited:", v)
```

This prints the path and the visited nodes.

For the gray graph in the problem the path & visited are

BFS path: ['A', 'B', 'E', 'H']
BFS visited: ['A', 'B', 'C', 'D', 'E', 'F', 'G']